

CowControl — Cómo la app procesa los tags

Para Germaioni — por qué no se veían tus tags, y el contrato exacto app ↔ hardware

1. Resumen rápido: por qué no se te veían los tags

Con los videos que mandaste se confirma algo importante: **tu ESP32 funciona bien de punta a punta**. En el Serial Monitor se ve que se conecta a WiFi, se conecta a Supabase (status 200), lee una tarjeta real (UID leído: E053B619) y la guarda (UID guardado — E053B619). Y en el Table Editor de Supabase se ve esa misma fila insertada en datos_lectura. O sea: el problema **no está en tu hardware ni en la base de datos** — el tag llega bien.

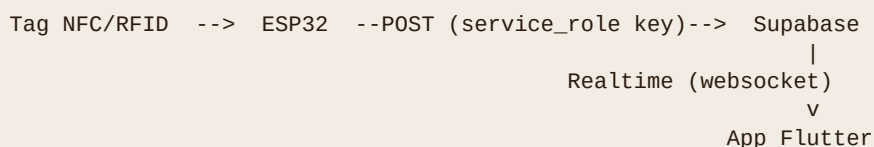
El problema estaba del lado de la app, y ya está corregido. Hasta el 2026-09-13, cuando llegaba una lectura cuyo rfid_uid no coincidía con ningún animal ya cargado en datos_animales, la Pantalla 3 (conteo en vivo) simplemente la ignoraba — una línea de código hacía return y esa lectura desaparecía sin dejar rastro en pantalla. Como tus tags de prueba no estaban pre-cargados como animales, eso es exactamente lo que te pasó.

Desde el 2026-09-13 eso cambió: cualquier tag que no matchea un animal ya no se descarta, aparece en una sección aparte llamada **«Sin identificar»** dentro de la misma Pantalla 3. Tocando esa fila se abre un diálogo para elegirle una categoría (vaca/toro/ternero/ternera/vaquillona), y ahí recién se crea el animal en Supabase. Sección 4 de este documento tiene el código exacto.

Importante para volver a probar: el APK que compilamos hoy (16/9, versionCode 2, versionName 1.0.1) ya tiene este fix. Si lo recibiste por WhatsApp y seguís viendo el comportamiento viejo, puede ser un problema de caché de WhatsApp (ya nos pasó con otro compañero): adjuntá el archivo de nuevo desde el buscador de archivos del celular en vez de reenviar un mensaje viejo, y si tenés dudas revisá la versión instalada con `adb shell dumpsys package com.cowcontrol.app_cowcontrol` (tiene que decir versionCode=2).

2. Arquitectura completa (para tener el mapa entero)

El principio de todo el proyecto es que **la app nunca habla directo con tu ESP32** — todo pasa por Supabase, que hace de único «servidor» (base de datos + API + autenticación + tiempo real):



Tu ESP32 hace dos cosas nada más: (1) cuando lee un tag, hace un INSERT a datos_lectura; (2) cada tantos segundos, consulta (polling) la tabla conteos para saber si hay un conteo activo en tu campo. La app, del otro lado, se suscribe a datos_lectura por Supabase Realtime (websocket) — apenas Postgres registra tu INSERT, a la app le llega la novedad sola, sin refrescar nada.

3. El contrato exacto que la app espera de tu firmware

3.1 Insertar una lectura — tabla datos_lectura:

Columnas reales (¡ojo, ya NO existe id_animal — se sacó en 2026-08-31 porque nunca se usaba; la app matchea por rfid_uid al momento, no antes!):

```
id          uuid, la genera Supabase sola
rfid_uid    text  -> el UID que lee tu RC522/ESP32
campo_id    uuid  -> fijo por instalación (ver 3.3)
fecha_hora  timestampz, default now()
observaciones text, opcional
```

POST de ejemplo (mismo formato que Guia_Raspberry_Pi-CowControl.pdf):

POST https://jucndmefmewkjalqnrsv.supabase.co/rest/v1/datos_lectura

headers:

```
apikey: <SERVICE_ROLE_KEY>
Authorization: Bearer <SERVICE_ROLE_KEY>
Content-Type: application/json
```

body:

```
{"rfid_uid": "E053B619", "campo_id": "<uuid-de-tu-campo>"}
```

La service_role key bypassa RLS por completo — por eso el ESP32 puede escribir sin login. **Nunca la pongas en un repo público ni en la app.**

3.2 Consultar el estado del conteo — tabla conteos:

Columnas: id (uuid), campo_id (uuid), fecha (timestampz), estado (text)

CORRECCION IMPORTANTE sobre lo que tenías anotado:
el CHECK real de "estado" tiene 3 valores exactos:

```
'en_curso' | 'pausado' | 'finalizado'
```

NO existe el valor 'activo' — si tu firmware está preguntando por ese string exacto, nunca va a matchear nada. Este malentendido ya se había detectado el 2026-08-31 (prompts.md sección 16.2) pero vale la pena confirmarlo de nuevo por si tu firmware nuevo de ESP32 lo heredó.

GET https://jucndmefmewkjalqnrsv.supabase.co/rest/v1/conteos?campo_id=eq.<tu-campo-id>&estado=eq.e

3.3 El campo_id es fijo por instalación, no dinámico

A diferencia de la app (que sabe el campo del productor porque está logueado), tu ESP32 no tiene login — necesita tener el campo_id (un UUID) escrito como constante en el firmware, una sola vez, cuando instalás el equipo en un campo determinado. Podés conseguirlo consultando en Supabase:

```
SELECT id, nombre_campo FROM datos_campo;
```

4. Código real del lado de la app

Esto es lo que corre en la Pantalla 3 (conteo en vivo) hoy mismo. Te lo paso completo para que, si necesitás entender o ajustar algo, puedas pegarle esto a tu propio Claude y que entienda el contrato exacto sin adivinar nada.

4.1 Cómo se decide si un tag es conocido o "sin identificar" —

lib/screens/conteo_screen.dart:

```

void _marcarLectura(Lectura lectura) {
  Animal? animalCoincidente;
  for (final animal in _animales) {
    if (animal.rfidUid == lectura.rfidUid) {
      animalCoincidente = animal;
      break;
    }
  }
  if (animalCoincidente == null) {
    if (!_sinIdentificar.contains(lectura.rfidUid)) {
      setState(() => _sinIdentificar.add(lectura.rfidUid));
    }
    return;
  }
  setState(() => _idsDetectados.add(animalCoincidente!.id));
}

Future<void> _asignarCategoria(String rfidUid) async {
  final categoria = await showDialog<String>(
    context: context,
    builder: (_) => const _DialogoCategoria(),
  );
  if (categoria == null) return;
  final nuevoAnimal = await _animalesService.crearAnimal(
    rfidUid: rfidUid,
    campoId: widget.campo.id,
    categoria: categoria,
  );
  setState(() {
    _sinIdentificar.remove(rfidUid);
    _animales.add(nuevoAnimal);
    _idsDetectados.add(nuevoAnimal.id);
  });
}

```

4.2 Dar de alta el animal nuevo —

lib/services/animales_service.dart:

```

Future<Animal> crearAnimal({
  required String rfidUid,
  required String campoId,
  required String categoria,
}) async {
  final fila = await _client
    .from('datos_animales')
    .insert({
      'rfid_uid': rfidUid,
      'campo_id': campoId,
      'sexo': categoria,
      'estado_vital': 'vivo',
    })
    .select()
    .single();
  return Animal.fromMap(fila);
}

```

4.3 Ciclo de vida del conteo — lib/services/conteos_service.dart (completo):

```

class ConteosService {
  final _client = SupabaseService.client;

  // Si ya hay un conteo en pausa para este campo, lo reanuda.
  // Si no hay ninguno, crea uno nuevo en estado "en_curso".
  Future<String> iniciarOReanudar(String campoId) async {
    final pausado = await _client.from('conteos').select('id')
      .eq('campo_id', campoId).eq('estado', 'pausado').maybeSingle();
    if (pausado != null) {
      final id = pausado['id'] as String;
      await reanudar(id);
      return id;
    }
    final nuevo = await _client.from('conteos').insert({
      'campo_id': campoId,
      'fecha': DateTime.now().toUtc().toIso8601String(),
      'estado': 'en_curso',
    }).select('id').single();
    return nuevo['id'] as String;
  }

  Future<void> pausar(String conteoId) => _client.from('conteos')
    .update({'estado': 'pausado'}).eq('id', conteoId);

  Future<void> reanudar(String conteoId) => _client.from('conteos')
    .update({'estado': 'en_curso'}).eq('id', conteoId);

  Future<void> finalizar(String conteoId) => _client.from('conteos')
    .update({'estado': 'finalizado'}).eq('id', conteoId);
}

```

4.4 Suscripción en tiempo real — lib/services/lecturas_service.dart:

```

RealtimeChannel escucharNuevasLecturas({
  required String campoId,
  required void Function(Lectura) alRecibirLectura,
}) {
  final canal = _client.channel('lecturas_campo_$campoId');
  canal.onPostgresChanges(
    event: PostgresChangeEvent.insert,
    schema: 'public',
    table: 'datos_lectura',
    filter: PostgresChangeFilter(
      type: PostgresChangeFilterType.eq,
      column: 'campo_id',
      value: campoId,
    ),
    callback: (payload) {
      alRecibirLectura(Lectura.fromMap(payload.newRecord));
    },
  ).subscribe();
  return canal;
}

```

5. Si además vas a meterte con la función de cámara

Keila ya está armando esa parte por su lado (un script en Python + OpenCV que corre en la netbook con la webcam, grabando y subiendo a Supabase Storage) — tiene su propio documento con el detalle técnico completo. Para no pisarse el trabajo: si vos avanzás algo de cámara, coordinen antes quién toca qué. Lo que sí te corresponde a vos de lleno es todo lo de la sección 3 de este documento (el contrato ESP32 ↔ Supabase) — mientras sigas esos nombres de tabla/columna y

esos 3 valores de estado, cualquier otra pieza del sistema (app, cámara) va a poder engancharse sin romper nada de tu lado.

6. Checklist para volver a probar

- Confirmar que el APK instalado es el de hoy (versionCode 2 / versionName 1.0.1).
- Cargar al menos un conteo "en_curso" desde la app antes de que el ESP32 empiece a mandar lecturas (si no hay conteo activo, tu polling nunca va a encontrar nada que hacer, aunque el POST a datos_lectura funcione igual).
- Pasar un tag nuevo (que no esté cargado como animal) y confirmar que aparece en la sección "Sin identificar" de la Pantalla 3, no que desaparece.
- Tocarlo, elegirle una categoría, y confirmar que pasa a la lista de animales detectados.
- Si algo no aparece, lo primero a revisar es que el string de "estado" que manda tu firmware sea exactamente 'en_curso' (sin mayúsculas ni espacios) — Postgres compara texto exacto.